

Tests de validation

Les tests suivants ont pour objectif de valider le bon fonctionnement de l'ensemble des mécanismes de sécurité mis en œuvre dans le laboratoire, notamment les règles de **filtrage du pare-feu**, la gestion des **flux réseau entre les différentes zones**, les mécanismes de **traduction d'adresses (NAT)**, ainsi que la **détection d'intrusion** avec **Suricata** et la remontée des alertes associées dans **Wazuh**.

Ces **scénarios** permettent également de vérifier le cheminement des **événements** de sécurité à travers les différents composants de l'architecture, depuis la génération d'un **trafic réseau** jusqu'à sa **détection**, sa **journalisation** et son affichage dans l'interface de **supervision**.

Afin de faciliter la compréhension des différents scénarios de test, l'architecture réseau du laboratoire est rappelée ci-dessous :

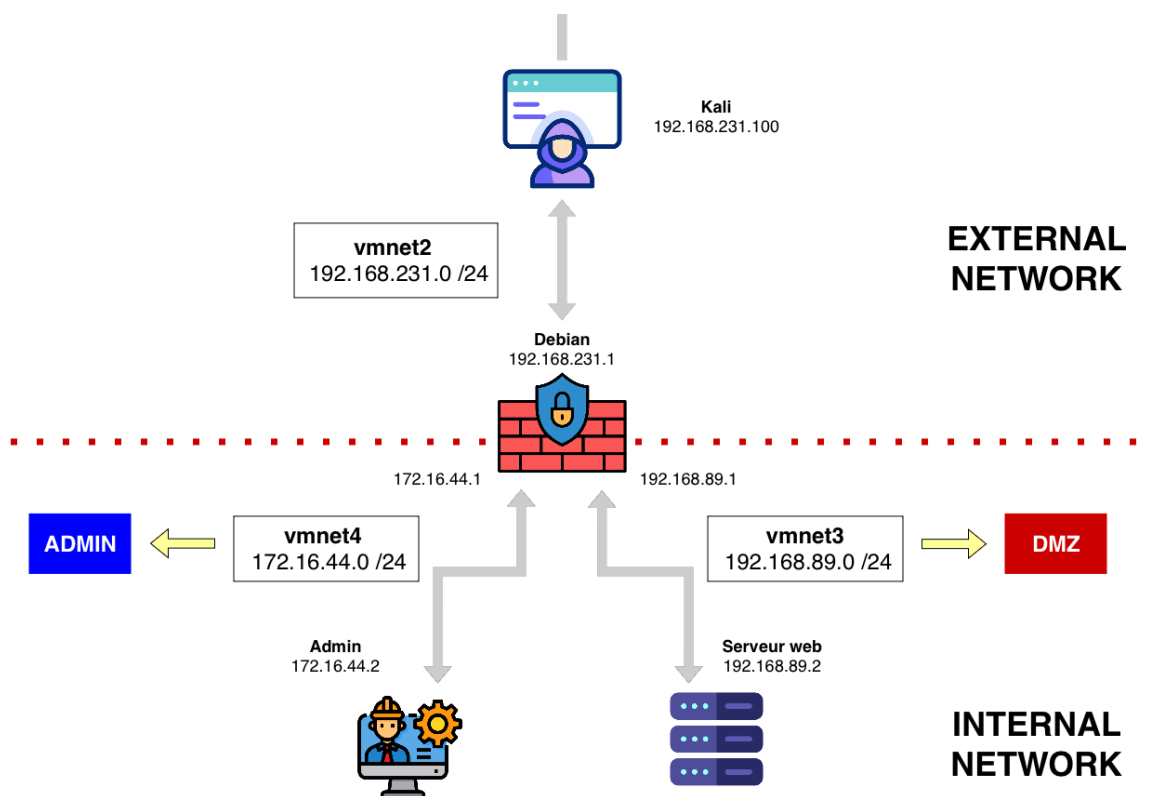


Schéma - linux-network-security-lab

Sommaire

Validation des journaux du pare-feu et des alertes Wazuh	4
Test 1 : Validation de la règle correspondant au préfixe CT-ERR_FROM-EXTERNAL_ICMP-REP_	4
Test 2 : Validation de la règle correspondant au préfixe CT-ERR_FROM-EXTERNAL_HTTP-REP_	6
Test 3 : Validation de la règle correspondant au préfixe CT-ERR_FROM-DMZ-TO-ADMIN_HTTP-REP_	7
Test 4 : Validation de la règle correspondant au préfixe CT-ERR_FROM-DMZ-TO-ADMIN_ICMP-REP_	9
Test 5 : Validation de la règle correspondant au préfixe CT-ERR_FROM-DMZ-TO-ADMIN_SSH-REP_	10
Test 6 : Validation de la règle correspondant au préfixe DENY_FROM-EXTERNAL-TO-ADMIN_HTTP-REQ_	11
Test 7 : Validation de la règle correspondant au préfixe DENY_FROM-DMZ-TO-ADMIN_HTTP-REQ_	13
Test 8 : Validation de la règle correspondant au préfixe DENY_FROM-EXTERNAL-TO-ADMIN_SSH-REQ_	14
Test 9 : Validation de la règle correspondant au préfixe DENY_FROM-DMZ-TO-ADMIN_SSH-REQ_	15
Test 10 : Validation de la règle correspondant au préfixe DENY_FROM-EXTERNAL-TO-ADMIN_ICMP-REQ_	16
Test 11 : Validation de la règle correspondant au préfixe DENY_FROM-EXTERNAL-TO-DMZ_ICMP-REQ_	17
Test 12 : Validation de la règle correspondant au préfixe DENY_FROM-DMZ-TO-ADMIN_ICMP-REQ_	19
Test 13 : Validation de la règle correspondant au préfixe DENY_FROM-EXTERNAL-TO-DMZ_SSH-REQ_	20
Validation de la détection d'intrusion par Suricata et des alertes Wazuh associées	21
Test 14 : Validation de la détection Suricata d'un scan réseau via une anomalie ICMP	22
Test 15 : Validation de la détection Suricata avec la règle ET OPEN 210048.....	24

Validation des fonctionnalités réseau du pare-feu	26
Test 16 : Validation du DNAT – Accès HTTP depuis Kali vers le serveur Web.....	26
Test 17 : Validation du SNAT – Accès HTTP depuis le serveur Web vers un service externe	27
Test 18 : Connexion SSH depuis la machine d'administration vers le pare-feu	28
Test 18 : Connexion SSH depuis la machine d'administration vers le serveur Web	29
Test 20 : Accès HTTP depuis la machine d'administration vers le serveur Web	29

Validation des journaux du pare-feu et des alertes

Wazuh

Afin de valider la chaîne de collecte et de traitement des journaux du pare-feu, différents scénarios sont générés à l'aide de **Scapy** depuis la machine émettrice.

Pour chaque scénario correspondant à un préfixe sur les règles de logs du pare-feu, les étapes de validation sont les suivantes :

- génération d'un paquet avec **Scapy** ;
- vérification de la journalisation par le pare-feu et de la collecte du journal par **rsyslog** dans le fichier **kern.log** ;
- vérification de la remontée de l'événement dans **Wazuh** sous forme d'alerte.

Test 1 : Validation de la règle correspondant au préfixe CT-ERR_FROM-EXTERNAL_ICMP-REP_

Description : réponse **ICMP** provenant de la zone externe et échouant au suivi de connexion (**Connection Tracking**).

Le trafic est généré depuis la machine **Kali**, située sur le réseau externe **vmnet2** (**192.168.231.0/24**), à l'aide de **Scapy**.

```

Scapy 2.7.01
Session Actions Éditer Vue Aide

(kali@kali)-[~]
$ sudo scapy
INFO: Can't import PyX. Won't be able to use psdump() or pdfdump().

      aSPY//YASa
      apyyyyCY////////YCca
      sY////////YSpcs  scpCY//Pp
      ayp ayyyyyySCP//Pp  syY//C
      AYAsAYYYYYYYY//Ps  cY//S
      pCCCCY//p  cSSps y//Y
      SPPPP//a  pP//AC//Y
      A//A  cyP///C
      p///Ac  sC///a
      P///YCpc  A//A
      scccccp///pSP///p  p//Y
      sY////////y  caa  S//P
      cayCyayP//Ya  pY/Ya
      sY/PsY////////YCc  aC//Yp
      sc  sccaCY//PCypaapyCP//YSs
      spCPY////////YPSps
      ccaacs

Welcome to Scapy
Version 2.7.01
https://github.com/secdev/scapy
Have fun!
I'll be back.
-- Python 2

using IPython 9.11.0
>>> send(IP(dst="192.168.89.2")/ICMP(type="echo-reply", id=1234, seq=1))
.
Sent 1 packets.
>>>
  
```

Le journal généré par le pare-feu est ensuite vérifié afin de confirmer que la règle de journalisation a bien été déclenchée

```
radlab@debian:~$ sudo tail -f -n 0 /var/log/kern.log
2026-07-28T19:47:49.698912+02:00 debian kernel: CT-ERR_FROM-EXTERNAL_ICMP-REP_IN=enp2s0 OUT=enp26s0 MAC=00:50:56:3a:e4:ba:00:0c:29:a3:9c:52:08:00 SRC=192.168.231.100 DST=192.168.89.2 LEN=28 TOS=0x00 PREC=0x00 TTL=63 ID=1 PROTO=ICMP TYPE=0 CODE=0 ID=1234 SEQ=1
```

L'événement est ensuite recherché dans **Wazuh**, depuis le module **Threat Hunting** de l'agent installé sur le pare-feu (**machine Debian**).

timestamp	agent.name	rule.description	rule.level	rule.id
Jul 28, 2026 @ 19:47:51.192	firewall-debian	Failed ICMP connection tracking validationfrom external host	7	100051
Jul 28, 2026 @ 19:47:51.192	firewall-debian	Failed ICMP connection tracking validationfrom external host	7	100051
Jul 28, 2026 @ 19:47:51.195	firewall-debian	Successful sudo to ROOT executed	3	5402

La description de l'alerte détectée, correspond à celle que nous avons définie dans le fichier **local_rules.xml**

Failed ICMP connection tracking validationfrom external host

En inspectant les détails de l'événement (via l'icône de loupe), il est possible de consulter les informations brutes du journal.

Document Details		View surrounding documents	View single document
Table	JSON		
index	wazuh-alerts-4.x-2026.07.28		
agent.id	001		
agent.ip	172.16.44.1		
agent.name	firewall-debian		
decoder.name	kernel		
full_log	2026-07-28T19:47:49.698912+02:00 debian kernel: CT-ERR_FROM-EXTERNAL_ICMP-REP_IN=enp2s0 OUT=enp26s0 MAC=00:50:56:3a:e4:ba:00:0c:29:a3:9c:52:08:00 SRC=192.168.231.100 DST=192.168.89.2 LEN=28 TOS=0x00 PREC=0x00 TTL=63 ID=1 PROTO=ICMP TYPE=0 CODE=0 ID=1234 SEQ=1		
id	1785260871.216847		
input.type	log		
location	/var/log/kern.log		
manager.name	radlab-VMware20-1		
predecoder.program_name	kernel		
predecoder.timestamp	2026-07-28T19:47:49.698912+02:00		
rule.description	Failed ICMP connection tracking validationfrom external host		

Le préfixe **CT-ERR_FROM-EXTERNAL_ICMP-REP_**, ajouté par le pare-feu lors de la journalisation, est bien présent dans le message. Cette observation confirme le bon fonctionnement de la chaîne de traitement, depuis la génération du journal par le pare-feu jusqu'à la construction de l'alerte et l'affichage dans **Wazuh**.

```
full_log 2026-07-28T19:47:49.698912+02:00 debian kernel: CT-ERR_FROM-EXTERNAL_ICMP-REP_IN=enp2s0 OUT=enp26s0 MAC=00:50:56:3a:e4:ba:00:0c:29:a3:9c:52:08:00 SRC=192.168.231.100 DST=192.168.89.2 LEN=28 TOS=0x00 PREC=0x00 TTL=63 ID=1 PROTO=ICMP TYPE=0 CODE=0 ID=1234 SEQ=1
```

Test 2 : Validation de la règle correspondant au préfixe CT-ERR_FROM-EXTERNAL_HTTP-REP_

Description : réponse **HTTP** provenant de la zone externe et échouant au suivi de connexion (**Connection Tracking**).

Le trafic est généré depuis la machine **Kali**, située sur le réseau externe **vmnet2** (**192.168.231.0/24**), afin de déclencher la règle de journalisation correspondante sur le pare-feu.

```

Scapy 2.7.01
Session Actions Éditer Vue Aide

sY/////YSpCs scpCY//Pp | Welcome to Scapy
ayp ayyyyyySCP//Pp syY//C | Version 2.7.01
AYAsAYYYYYYYY//Ps cY//S | https://github.com/secdev/scapy
pCCCCY//p cSSps y//Y | Have fun!
SPPPP///a pP///AC//Y | Craft me if you can.
A//A cyP///C | -- IPv6 layer
p///Ac sC///a
P///YCpc A//A
scccccp///pSP///p p//Y
sY/////////y caa S//P
cayCyayP//Ya pY/Ya
sY/PSY///YCc aC/Yp
sc sccaCY//PCypaapyCP//Yss
spCPY/////////YPSps
ccaacs

using IPython 9.11.0
Tip: Run your doctests from within IPython for development and debugging. The
special %doctest_mode command toggles a mode where the prompt, output and ex
ceptions display matches as closely as possible that of the default Python in
terpreter.
>>> from scapy.all import *
... :
... : send(
... : IP(src="192.168.231.100", dst="172.16.44.2") / TCP(sport=80,dport=50
... : 000))
.
Sent 1 packets.
>>>

```

Le journal produit par le pare-feu est ensuite vérifié afin de confirmer que la règle associée au préfixe **CT-ERR_FROM-EXTERNAL_HTTP-REP_** a bien été déclenchée.

```

radiolab@debian:~$ sudo tail -f -n 0 /var/log/kern.log
2026-07-29T13:27:28.146593+02:00 debian kernel: CT-ERR_FROM-EXTERNAL_HTTP-REP_IN=enp2s0 OUT=enp3s0 MAC=00:50:56:3a:e4:ba:00:0c:29:a3:9c:52:00:00 SRC=192.168.231.100 DST=172.16.44.2 LEN=40 TOS=0x00 PREC=0x00 TTL=63 ID=1 PROTO=TCP SPT=80 DPT=50000 WINDOW=6192 RES=0x00 SYN URG=0

```

L'événement est ensuite recherché dans **Wazuh**, depuis le module Threat Hunting de l'agent installé sur le pare-feu (machine **Debian**).

La description de l'alerte personnalisée associée à cette règle correspond bien à celle définie dans le fichier **local_rules.xml**.

Jul 29, 2026 @ 13:13:19.315 - Jul 29, 2026 @ 13:28:19.315				
Export Formatted	Reset view	691 available fields	Columns	Density
1 fields sorted	Full screen			
timestamp	agent.name	rule.description	rule.level	rule.id
Jul 29, 2026 @ 13:27:28.516	firewall-debian	Failed HTTP connection tracking validation from external host	7	100052

En consultant les détails de l'événement, il est possible d'observer le message brut généré par le pare-feu.

Document Details [View surrounding documents](#) [View single document](#)

Table	JSON
<code>_index</code>	wazuh-alerts-4.x-2026.07.29
<code>agent.id</code>	001
<code>agent.ip</code>	172.16.44.1
<code>agent.name</code>	firewall-debian
<code>decoder.name</code>	kernel
<code>full_log</code>	Jul 29 11:27:28 debian kernel: CT-ERR_FROM-EXTERNAL_HTTP-REP_IN=enp2s0 OUT=enp3s0 MAC=00:50:56:3a:e4:ba:00:0c:29:a3:9c:52:08:00 SRC=192.168.231.100 DST=172.16.44.2 LEN=40 TOS=0x00 PREC=0x00 TTL=63 ID=1 PROTO=TCP SPT=80 DPT=50000 WINDOW=8192 RES=0x00 SYN URG=0
<code>id</code>	1785324448.58064
<code>input.type</code>	log
<code>location</code>	journald
<code>manager.name</code>	radlab-VMware20-1
<code>predecoder.hostname</code>	debian
<code>predecoder.program_name</code>	kernel
<code>predecoder.timestamp</code>	Jul 29 11:27:28
<code>rule.description</code>	Failed HTTP connection tracking validation from external host

Le préfixe **CT-ERR_FROM-EXTERNAL_HTTP-REP_**, ajouté par le pare-feu lors de la journalisation, est bien présent dans le message. Cette observation confirme le bon fonctionnement de la chaîne de traitement, depuis la génération du journal par le pare-feu jusqu'à la création de l'alerte personnalisée et son affichage dans **Wazuh**.

Test 3 : Validation de la règle correspondant au préfixe CT-ERR_FROM-DMZ-TO-ADMIN_HTTP-REP_

Description : réponse **HTTP** provenant de la zone **DMZ** vers la zone d'administration et échouant au suivi de connexion (**Connection Tracking**).

Le trafic est généré depuis le serveur Web, situé dans la zone **DMZ vmnet3 (192.168.89.0/24)**, afin de déclencher la règle de journalisation correspondante sur le pare-feu.

```

radlab@radlabserv:~$ sudo scapy
INFO: Can't import PyX. Won't be able to use psdump() or pdfdump().

      aSPY//YASa
    apyyyyCY////////YCa
  sY////////YSpcs  scpCY//Pp
ayp ayyyyyyySCP//Pp      syY//C
AYAsAYYYYYYYY//Ps      cY//S
  pCCCCY//p      cSSps y//Y
  SPPPP//a      pP///AC//Y
    A//A      cyP///C
  p///Ac      sC///a
  P///YCpc      A//A
  scccccp///pSP///p      p//Y
  SY////////y caa      S//P
  cayCyayP//Ya      pY/Ya
  sY/PSY///YCC      aC//Yp
  sc  sccaCY//PCypaapyCP//YSs
      spCPY////////YPSps
      ccaacs

using IPython 8.20.0
>>> send(IP(src="192.168.89.2", dst="172.16.44.2") / TCP(sport=80, dport=5000, flags="SA", seq=1000, ack=1))
.
Sent 1 packets.
>>> _

```

Le journal produit par le pare-feu est ensuite vérifié afin de confirmer que la règle associée au préfixe **CT-ERR_FROM-DMZ-TO-ADMIN_HTTP-REP_** a bien été déclenchée.

```

radlab@debian:~$ sudo tail -f -n 0 /var/log/kern.log
2026-07-28T20:14:22.687053+02:00 debian kernel: CT-ERR_FROM-DMZ-TO-ADMIN_HTTP-REP_IN=enp26s0 OUT=enp3s0 MAC=00:0c:29:32:e3:d8:00:0c:29:3e:56:fd:08:00 SRC=192.16
8.89.2 DST=172.16.44.2 LEN=40 TOS=0x00 PREC=0x00 TTL=63 ID=1 PROTO=TCP SPT=80 DPT=5000 WINDOW=8192 RES=0x00 ACK SYN URGP=0

```

L'événement est ensuite recherché dans **Wazuh**, depuis le module **Threat Hunting** de l'agent installé sur le pare-feu (machine **Debian**).

La description de l'alerte personnalisée associée à cette règle correspond bien à celle définie dans le fichier **local_rules.xml**.

timestamp	agent.name	rule.description	rule.level	rule.id
Jul 28, 2026 @ 19:59:17.691	firewall-debian	Failed HTTP connexion tracking validation from DMZ to Admin host	7	100053

En consultant les détails de l'événement, il est possible d'observer le message brut généré par le pare-feu.

Document Details

View surrounding documents

View single document

Help

Table JSON

_index	wazuh-alerts-4.x-2026.07.28
agent.id	001
agent.ip	172.16.44.1
agent.name	firewall-debian
decoder.name	kernel
full_log	Jul 28 18:14:22 debian kernel: CT-ERR_FROM-DMZ-TO-ADMIN_HTTP-REP_IN=enp26s0 OUT=enp3s0 MAC=00:0c:29:32:e3:d8:00:0c:29:3e:56:fd:08:00 SRC=192.168.89.2 DST=172.16.44.2 LEN=40 TOS=0x00 PREC=0x00 TTL=63 ID=1 PROTO=TCP SPT=80 DPT=5000 WINDOW=8192 RES=0x00 ACK SYN URGP=0
id	1785261557.221032
input.type	log
location	journald
manager.name	radlab-VMware20-1
predecoder.hostname	debian
predecoder.program_name	kernel

Le préfixe **CT-ERR_FROM-DMZ-TO-ADMIN_HTTP-REP_**, ajouté par le pare-feu lors de la journalisation, est bien présent dans le message. Cette observation confirme le bon fonctionnement de la chaîne de traitement, depuis la génération du journal par le pare-feu jusqu'à la création de l'alerte personnalisée et son affichage dans **Wazuh**.

Test 4 : Validation de la règle correspondant au préfixe CT-ERR_FROM-DMZ-TO-ADMIN_ICMP-REP_

Description : réponse **ICMP** provenant de la zone **DMZ** vers la zone d'administration et échouant au suivi de connexion (**Connection Tracking**).

```

>>> send(IP(src="192.168.89.2", dst="172.16.44.2") / ICMP(type="echo-reply", id=1000, seq=1))
Sent 1 packets.

```

Le trafic est généré depuis le serveur Web, situé dans la zone **DMZ vmnet3 (192.168.89.0/24)**, afin de déclencher la règle de journalisation correspondante sur le pare-feu.

```

radlab@debian:~$ sudo tail -f -n 0 /var/log/kern.log
2026-07-28T20:17:54.888799+02:00 debian kernel: CT-ERR_FROM-DMZ-TO-ADMIN_ICMP-REP_IN=enp26s0 OUT=enp3s0 MAC=00:0c:29:32:e3:d8:00:0c:29:3e:56:fd:08:00 SRC=192.168.89.2 DST=172.16.44.2 LEN=28 TOS=0x00 PREC=0x00 TTL=63 ID=1 PROTO=ICMP TYPE=0 CODE=0 ID=1000 SEQ=1

```

L'événement est ensuite recherché dans **Wazuh**, depuis le module **Threat Hunting** de l'agent installé sur le pare-feu (machine **Debian**).

La description de l'alerte personnalisée associée à cette règle correspond bien à celle définie dans le fichier **local_rules.xml**.

timestamp	agent.name	rule.description	rule.level	rule.id
Jul 28, 2026 @ 20:02:49.799	firewall-debian	Failed ICMP connexion tracking validation from DMZ to Admin host	7	100054

En consultant les détails de l'événement, il est possible d'observer le message brut généré par le pare-feu.

Document Details

[View surrounding documents](#)
[View single document](#)

Table JSON

t _index	wazuh-alerts-4.x-2026.07.28
t agent.id	001
t agent.ip	172.16.44.1
t agent.name	firewall-debian
t decoder.name	kernel
t full_log	Jul 28 18:17:54 debian kernel: CT-ERR_FROM-DMZ-TO-ADMIN_ICMP-REP IN=enp26s0 OUT=enp3s0 MAC=00:0c:29:32:e3:d8:00:0c:29:3e:56:fd:08:00 SRC=192.168.89.2 DST=172.16.44.2 LEN=28 TOS=0x00 PREC=0x00 TTL=63 ID=1 PROTO=ICMP TYPE=0 CODE=0 ID=1000 SEQ=1
t id	1785261769.223342
t input.type	log
t location	journald
t manager.name	radlab-VMware20-1
t predecoder.hostname	debian

Le préfixe **CT-ERR_FROM-DMZ-TO-ADMIN_ICMP-REP_**, ajouté par le pare-feu lors de la journalisation, est bien présent dans le message. Cette observation confirme le bon fonctionnement de la chaîne de traitement, depuis la génération du journal par le pare-feu jusqu'à la création de l'alerte personnalisée et son affichage dans Wazuh.

Test 5 : Validation de la règle correspondant au préfixe CT-ERR_FROM-DMZ-TO-ADMIN_SSH-REP_

Description : réponse **SSH** provenant de la zone **DMZ** vers la zone d'administration et échouant au suivi de connexion (**Connection Tracking**).

Le trafic est généré depuis le serveur Web, situé dans la zone **DMZ vmnet3 (192.168.89.0/24)**, afin de déclencher la règle de journalisation correspondante sur le pare-feu.

```
>>>
>>> send(IP(src="192.168.89.2", dst="172.16.44.2") / TCP(sport=22, dport=5000, flags="SA", seq=1000, ack=1))
Sent 1 packets.
>>> _
```

Le journal produit par le pare-feu est ensuite vérifié afin de confirmer que la règle associée au préfixe **CT-ERR_FROM-DMZ-TO-ADMIN_SSH-REP_** a bien été déclenchée.

```
radiolab@debian:~$ sudo tail -f -n 0 /var/log/kern.log
2026-07-28T20:22:51.577585+02:00 debian kernel: CT-ERR_FROM-DMZ-TO-ADMIN_SSH-REP_IN=enp26s0 OUT=enp3s0 MAC=00:0c:29:32:e3:d8:00:0c:29:3e:56:fd:08:00 SRC=192.168.89.2 DST=172.16.44.2 LEN=40 TOS=0x00 PREC=0x00 TTL=63 ID=1 PROTO=TCP SPT=22 DPT=5000 WINDOW=8192 RES=0x00 ACK SYN URGP=0
```

L'événement est ensuite recherché dans **Wazuh**, depuis le module **Threat Hunting** de l'agent installé sur le pare-feu (machine **Debian**).

La description de l'alerte personnalisée associée à cette règle correspond bien à celle définie dans le fichier **local_rules.xml**.

timestamp	agent.name	rule.description	rule.level	rule.id
Jul 28, 2026 @ 20:22:51.692	firewall-debian	Failed SSH connexion tracking validation from DMZ to Admin host	10	100055

En consultant les détails de l'événement, il est possible d'observer le message brut généré par le pare-feu.

Document Details

[View surrounding documents](#)
[View single document](#)

Table JSON

f _index	wazuh-alerts-4.x-2026.07.28
f agent.id	001
f agent.ip	172.16.44.1
f agent.name	firewall-debian
f decoder.name	kernel
f full_log	2026-07-28T20:22:51.577585+02:00 debian kernel: CT-ERR_FROM-DMZ-TO-ADMIN_SSH-REP_IN=enp26s0 OUT=enp3s0 MAC=00:0c:29:32:e3:d8:00:0c:29:3e:56:fd:08:00 SRC=192.168.89.2 DST=172.16.44.2 LEN=40 TOS=0x00 PREC=0x00 TTL=63 ID=1 PROTO=TCP SPT=22 DPT=5000 WINDOW=8192 RES=0x00 ACK SYN URGP=0
f id	1785262971.225625

Le préfixe **CT-ERR_FROM-DMZ-TO-ADMIN_SSH-REP_**, ajouté par le pare-feu lors de la journalisation, est bien présent dans le message. Cette observation confirme le bon fonctionnement de la chaîne de traitement, depuis la génération du journal par le pare-feu jusqu'à la création de l'alerte personnalisée et son affichage dans Wazuh.

Test 6 : Validation de la règle correspondant au préfixe **DENY_FROM-EXTERNAL-TO-ADMIN_HTTP-REQ_**

Description : requête **HTTP** provenant de la zone externe à destination de la zone d'administration.

Le trafic est généré depuis la machine Kali, située sur le réseau externe **vmnet2 (192.168.231.0/24)**, afin de vérifier l'application de la règle de blocage correspondante.

```
>>> send(
... : IP(src="192.168.231.100", dst="172.16.44.2") / TCP(dport=80,sport=50
... : 000,flags="SA",seq=1000,ack=1))
.
Sent 1 packets.
>>>
```

Le journal produit par le pare-feu est ensuite vérifié afin de confirmer que la règle associée au préfixe **DENY_FROM-EXTERNAL-TO-ADMIN_HTTP-REQ_** a bien été déclenchée.

```
radiab@debian:~$ sudo tail -f -n 0 /var/log/kern.log
2026-07-28T20:28:23.180075+02:00 debian kernel: DENY_FROM-EXTERNAL-TO-ADMIN_HTTP-REQ_IN=enp2s0 OUT=enp3s0 MAC=00:50:56:3a:e4:ba:00:0c:29:a3:9c:52:08:00 SRC=192.168.231.100 DST=172.16.44.2 LEN=40 TOS=0x00 PREC=0x00 TTL=63 ID=1 PROTO=TCP SPT=50000 DPT=80 WINDOW=8192 RES=0x00 ACK SYN URGP=0
```

L'événement est ensuite recherché dans **Wazuh**, depuis le module **Threat Hunting** de l'agent installé sur le pare-feu (machine **Debian**).

La description de l'alerte personnalisée associée à cette règle correspond bien à celle définie dans le fichier **local_rules.xml**.

timestamp	agent.name	rule.description	rule.level	rule.id
Jul 28, 2026 @ 20:28:23.817	firewall-debian	Http connection attempt from external to admin host	6	100056

En consultant les détails de l'événement, il est possible d'observer le message brut généré par le pare-feu.

Document Details

[View surrounding documents](#)
[View single document](#)

Table JSON

_index	wazuh-alerts-4.x-2026.07.28
agent.id	001
agent.ip	172.16.44.1
agent.name	firewall-debian
decoder.name	kernel
full_log	Jul 28 18:28:23 debian kernel: DENY_FROM-EXTERNAL-TO-ADMIN_HTTP-REQ_IN=enp2s0 OUT=enp3s0 MAC=00:50:56:3a:e4:ba:00:0c:29:a3:9c:52:08:00 SRC=192.168.231.100 DST=172.16.44.2 LEN=40 TOS=0x00 PREC=0x00 TTL=63 ID=1 PROTO=TCP SPT=50000 DPT=80 WINDOW=8192 RES=0x00 ACK SYN URGP=0
id	1785263303.227976

Le préfixe **DENY_FROM-EXTERNAL-TO-ADMIN_HTTP-REQ_**, ajouté par le pare-feu lors de la journalisation, est bien présent dans le message. Cette observation confirme le bon fonctionnement de la chaîne de traitement, depuis la génération du journal par le pare-feu jusqu'à la création de l'alerte personnalisée et son affichage dans **Wazuh**.

Test 7 : Validation de la règle correspondant au préfixe DENY_FROM-DMZ-TO-ADMIN_HTTP-REQ_

Description : requête **HTTP** provenant de la zone **DMZ** à destination de la zone d'administration.

Le trafic est généré depuis le serveur Web, situé dans la zone **DMZ vmnet3 (192.168.89.0/24)**, afin de déclencher la règle de journalisation correspondante.

```

Sent 1 packets.
>>> send(IP(src="192.168.89.2", dst="172.16.44.2") / TCP(dport=80, sport=5000, flags="S", seq=1000, ack=1))
.
Sent 1 packets.
>>>

```

Le journal produit par le pare-feu est ensuite vérifié afin de confirmer que la règle associée au préfixe **DENY_FROM-DMZ-TO-ADMIN_HTTP-REQ_** a bien été déclenchée.

```

radlab@debian:~$ sudo tail -f -n 0 /var/log/kern.log
2026-07-28T20:36:05.612177+02:00 debian kernel: DENY_FROM-DMZ-TO-ADMIN_HTTP-REQ_IN=enp26s0 OUT=enp3s0 MAC=00:0c:29:32:e3:d8:00:0c:29:3e:56:fd:08:00 SRC=192.168.89.2 DST=172.16.44.2 LEN=40 TOS=0x00 PREC=0x00 TTL=63 ID=1 PROTO=TCP SPT=5000 DPT=80 WINDOW=8192 RES=0x00 SYN URG=0

```

L'événement est ensuite recherché dans **Wazuh**, depuis le module **Threat Hunting** de l'agent installé sur le pare-feu (machine **Debian**).

timestamp	agent.name	rule.description	rule.level	rule.id
Jul 28, 2026 @ 20:33:53.942	firewall-debian	Http connection attempt from DMZ network to admin host	6	100061

En consultant les détails de l'événement, il est possible d'observer le message brut généré par le pare-feu.

Document Details

[View surrounding documents](#)
[View single document](#)

Table JSON

t _index	wazuh-alerts-4.x-2026.07.28
t agent.id	001
t agent.ip	172.16.44.1
t agent.name	firewall-debian
t decoder.name	kernel
t full_log	Jul 28 18:36:05 debian kernel: DENY_FROM-DMZ-TO-ADMIN_HTTP-REQ_IN=enp26s0 OUT=enp3s0 MAC=00:0c:29:32:e3:d8:00:0c:29:3e:56:fd:08:00 SRC=192.168.89.2 DST=172.16.44.2 LEN=40 TOS=0x00 PREC=0x00 TTL=63 ID=1 PROTO=TCP SPT=5000 DPT=80 WINDOW=8192 RES=0x00 SYN URG=0
t id	1785263633.230286
t input.type	log
t location	journalId
t manager.name	radlab-VMware20-1

La description de l'alerte personnalisée associée à cette règle correspond bien à celle définie dans le fichier **local_rules.xml**.

Le préfixe **DENY_FROM-DMZ-TO-ADMIN_HTTP-REQ_**, ajouté par le pare-feu lors de la journalisation, est bien présent dans le message. Cette observation confirme le bon fonctionnement de la chaîne de traitement, depuis la génération du journal par le pare-feu jusqu'à la création de l'alerte personnalisée et son affichage dans **Wazuh**.

Test 8 : Validation de la règle correspondant au préfixe **DENY_FROM-EXTERNAL-TO-ADMIN_SSH-REQ_**

Description : requête **SSH** provenant de la zone externe à destination de la zone d'administration.

Le trafic est généré depuis la machine **Kali**, située sur le réseau externe **vmnet2** (**192.168.231.0/24**), afin de déclencher la règle de journalisation correspondante.

```
>>> send(
... : IP(src="192.168.231.100", dst="172.16.44.2") / TCP(dport=22,sport=50
: 000,flags="S",seq=1000,ack=1))
.
```

Le journal produit par le pare-feu est ensuite vérifié afin de confirmer que la règle associée au préfixe **DENY_FROM-EXTERNAL-TO-ADMIN_SSH-REQ_** a bien été déclenchée.

```
radlab@debian:~$ sudo tail -f -n 0 /var/log/kern.log
2026-07-28T20:42:04.839662+02:00 debian kernel: DENY_FROM-EXTERNAL-TO-ADMIN_SSH-REQ_IN=enp2s0 OUT=enp3s0 MAC=00:50:56:3a:e4:ba:00:0c:29:a3:9c:52:08:00 SRC=192.168.231.100 DST=172.16.44.2 LEN=40 TOS=0x00 PREC=0x00 TTL=63 ID=1 PROTO=TCP SPT=50000 DPT=22 WINDOW=8192 RES=0x00 SYN URG=0
```

L'événement est ensuite recherché dans **Wazuh**, depuis le module **Threat Hunting** de l'agent installé sur le pare-feu (machine **Debian**).

La description de l'alerte personnalisée associée à cette règle correspond bien à celle définie dans le fichier **local_rules.xml**.

timestamp	agent.name	rule.description	rule.level	rule.id
Jul 28, 2026 @ 20:42:06.064	firewall-debian	SSH connection attempt from external to admin host	10	100057

En consultant les détails de l'événement, il est possible d'observer le message brut généré par le pare-feu.

Document Details

View surrounding documents

View single document

Table	JSON
<code>_index</code>	wazuh-alerts-4.x-2026.07.28
<code>agent.id</code>	001
<code>agent.ip</code>	172.16.44.1
<code>agent.name</code>	firewall-debian
<code>decoder.name</code>	kernel
<code>full_log</code>	2026-07-28T20:42:04.839662+02:00 debian kernel: DENY_FROM-EXTERNAL-TO-ADMIN_SSH-REQ IN=enp2s0 OUT=enp3s0 MAC=00:50:56:3a:e4:ba:00:0c:29:a3:52:08:00 SRC=192.168.231.100 DST=172.16.44.2 LEN=40 TOS=0x00 PREC=0x00 TTL=63 ID=1 PROTO=TCP SPT=50000 DPT=22 WINDOW=8192 RES=0x00 SYN URG=0
<code>id</code>	1785264126.232107
<code>input.type</code>	log
<code>location</code>	/var/log/kern.log
<code>manager.name</code>	radlab-VMware20-1
<code>predecoder.program_name</code>	kernel
<code>predecoder.timestamp</code>	2026-07-28T20:42:04.839662+02:00

Le préfixe **DENY_FROM-EXTERNAL-TO-ADMIN_SSH-REQ_**, ajouté par le pare-feu lors de la journalisation, est bien présent dans le message. Cette observation confirme le bon fonctionnement de la chaîne de traitement, depuis la génération du journal par le pare-feu jusqu'à la création de l'alerte personnalisée et son affichage dans **Wazuh**.

Test 9 : Validation de la règle correspondant au préfixe DENY_FROM-DMZ-TO-ADMIN_SSH-REQ_

Description : requête **SSH** provenant de la zone **DMZ** à destination de la zone d'administration.

Le trafic est généré depuis le serveur Web, situé dans la zone **DMZ vmnet3 (192.168.89.0/24)**, afin de déclencher la règle de journalisation correspondante.

```
>>> send(IP(src="192.168.89.2", dst="172.16.44.2") / TCP(dport=22, sport=5000, flags="S", seq=1000, ack=1))
Sent 1 packets.
>>> _
```

Le journal produit par le pare-feu est ensuite vérifié afin de confirmer que la règle associée au préfixe **DENY_FROM-DMZ-TO-ADMIN_SSH-REQ_** a bien été déclenchée.

```
radlab@debian:~$ sudo tail -f -n 0 /var/log/kern.log
2026-07-28T20:44:26.789699+02:00 debian kernel: DENY_FROM-DMZ-TO-ADMIN_SSH-REQ IN=enp2s0 OUT=enp3s0 MAC=00:0c:29:32:e3:d8:00:0c:29:3e:56:fd:08:00 SRC=192.168.89.2 DST=172.16.44.2 LEN=40 TOS=0x00 PREC=0x00 TTL=63 ID=1 PROTO=TCP SPT=5000 DPT=22 WINDOW=8192 RES=0x00 SYN URG=0
```

L'événement est ensuite recherché dans **Wazuh**, depuis le module **Threat Hunting** de l'agent installé sur le pare-feu (machine **Debian**).

La description de l'alerte personnalisée associée à cette règle correspond bien à celle définie dans le fichier **local_rules.xml**.

timestamp	agent.name	rule.description	rule.level	rule.id
Jul 28, 2026 @ 20:44:28.130	firewall-debian	SSH connection attempt from DMZ network to admin host	10	100062

En consultant les détails de l'événement, il est possible d'observer le message brut généré par le pare-feu.

Document Details

[View surrounding documents](#)
[View single document](#)

Table JSON

t _index	wazuh-alerts-4.x-2026.07.28
t agent.id	001
t agent.ip	172.16.44.1
t agent.name	firewall-debian
t decoder.name	kernel
t full_log	Jul 28 18:44:26 debian kernel: DENY_FROM-DMZ-TO-ADMIN SSH-REQ IN=enp26s0 OUT=enp3s0 MAC=00:0c:29:32:e3:d8:00:0c:29:3e:56:fd:08:00 SRC=192.168.89.2 DST=172.16.44.2 LEN=40 TOS=0x00 PREC=0x00 TTL=63 ID=1 PROTO=TCP SPT=5000 DPT=22 WINDOW=8192 RES=0x00 SYN URG=0
t id	1785264268.235827
t input.type	log
t location	journalld
t manager.name	radlab-VMware20-1
t predecoder.hostname	debian

Le préfixe **DENY_FROM-DMZ-TO-ADMIN_SSH-REQ_**, ajouté par le pare-feu lors de la journalisation, est bien présent dans le message. Cette observation confirme le bon fonctionnement de la chaîne de traitement, depuis la génération du journal par le pare-feu jusqu'à la création de l'alerte personnalisée et son affichage dans **Wazuh**.

Test 10 : Validation de la règle correspondant au préfixe DENY_FROM-EXTERNAL-TO-ADMIN_ICMP-REQ_

Description : requête **ICMP** provenant de la zone externe à destination de la zone d'administration.

Le trafic est généré depuis la machine **Kali**, située sur le réseau externe **vmnet2** (**192.168.231.0/24**), afin de déclencher la règle de journalisation correspondante.

```
(kali@kali)-[~]
$ ping 172.16.44.2
PING 172.16.44.2 (172.16.44.2) 56(84) bytes of data.

```


Le journal produit par le pare-feu est ensuite vérifié afin de confirmer que la règle associée au préfixe **DENY_FROM-EXTERNAL-TO-ADMIN_ICMP-REQ_** a bien été déclenchée.

```
radiab@debian:~$ sudo tail -f -n 0 /var/log/kern.log
2026-07-28T20:46:14.152678+02:00 debian kernel: DENY_FROM-EXTERNAL-TO-ADMIN_ICMP-REQ_IN=enp2s0 OUT=enp3s0 MAC=00:50:56:3a:e4:ba:00:0c:29:a3:9c:52:08:00 SRC=192.168.231.100 DST=172.16.44.2 LEN=84 TOS=0x00 PREC=0x00 TTL=63 ID=12486 DF PROTO=ICMP TYPE=8 CODE=0 ID=45471 SEQ=1
2026-07-28T20:46:15.183586+02:00 debian kernel: DENY_FROM-EXTERNAL-TO-ADMIN_ICMP-REQ_IN=enp2s0 OUT=enp3s0 MAC=00:50:56:3a:e4:ba:00:0c:29:a3:9c:52:08:00 SRC=192.168.231.100 DST=172.16.44.2 LEN=84 TOS=0x00 PREC=0x00 TTL=63 ID=12601 DF PROTO=ICMP TYPE=8 CODE=0 ID=45471 SEQ=2
```

L'événement est ensuite recherché dans **Wazuh**, depuis le module **Threat Hunting** de l'agent installé sur le pare-feu (machine **Debian**).

La description de l'alerte personnalisée associée à cette règle correspond bien à celle définie dans le fichier **local_rules.xml**.

timestamp	agent.name	rule.description	rule.level	rule.id
Jul 28, 2026 @ 20:47:10.214	firewall-debian	ICMP echo-request attempt from external to admin host	6	100058

En consultant les détails de l'événement, il est possible d'observer le message brut généré par le pare-feu.

Document Details		View surrounding documents	View single document
Table	JSON		
t _index	wazuh-alerts-4.x-2026.07.28		
t agent.id	001		
t agent.ip	172.16.44.1		
t agent.name	firewall-debian		
t decoder.name	kernel		
t full_log	Jul 28 18:47:09 debian kernel: DENY_FROM-EXTERNAL-TO-ADMIN_ICMP-REQ_IN=enp2s0 OUT=enp3s0 MAC=00:50:56:3a:e4:ba:00:0c:29:a3:9c:52:08:00 SRC=192.168.231.100 DST=172.16.44.2 LEN=84 TOS=0x00 PREC=0x00 TTL=63 ID=19419 DF PROTO=ICMP TYPE=8 CODE=0 ID=45471 SEQ=55		
t id	1785264430.288486		

Le préfixe **DENY_FROM-EXTERNAL-TO-ADMIN_ICMP-REQ_**, ajouté par le pare-feu lors de la journalisation, est bien présent dans le message. Cette observation confirme le bon fonctionnement de la chaîne de traitement, depuis la génération du journal par le pare-feu jusqu'à la création de l'alerte personnalisée et son affichage dans **Wazuh**.

Test 11 : Validation de la règle correspondant au préfixe **DENY_FROM-EXTERNAL-TO-DMZ_ICMP-REQ_**

Description : requête **ICMP** provenant de la zone externe à destination de la zone **DMZ**.

Le trafic est généré depuis la machine **Kali**, située sur le réseau externe **vmnet2** (**192.168.231.0/24**), afin de déclencher la règle de journalisation correspondante.

```
(kali@kali)-[~]
$ ping 192.168.89.2
PING 192.168.89.2 (192.168.89.2) 56(84) bytes of data.
```

Le journal produit par le pare-feu est ensuite vérifié afin de confirmer que la règle associée au préfixe **DENY_FROM-EXTERNAL-TO-DMZ_ICMP-REQ_** a bien été déclenchée.

```
radiolab@debian:~$ sudo tail -f -n 0 /var/log/kern.log
2026-07-28T20:51:39.742446+02:00 debian kernel: DENY_FROM-EXTERNAL-TO-DMZ_ICMP-REQ_IN=enp2s0 OUT=enp26s0 MAC=00:50:56:3a:e4:ba:00:0c:29:a3:9c:52:08:00 SRC=192.168.231.100 DST=192.168.89.2 LEN=84 TOS=0x00 PREC=0x00 TTL=63 ID=43080 DF PROTO=ICMP TYPE=8 CODE=0 ID=45472 SEQ=1
```

L'événement est ensuite recherché dans **Wazuh**, depuis le module **Threat Hunting** de l'agent installé sur le pare-feu (machine **Debian**).

La description de l'alerte personnalisée associée à cette règle correspond bien à celle définie dans le fichier **local_rules.xml**.

timestamp	agent.name	rule.description	rule.level	rule.id
Jul 28, 2026 @ 20:51:42.346	firewall-debian	ICMP echo-request attempt from external to DMZ network host	5	100060

En consultant les détails de l'événement, il est possible d'observer le message brut généré par le pare-feu.

Document Details

[View surrounding documents](#)
[View single document](#)

Table JSON

t _index	wazuh-alerts-4.x-2026.07.28
t agent.id	001
t agent.ip	172.16.44.1
t agent.name	firewall-debian
t decoder.name	kernel
t full_log	Jul 28 18:51:40 debian kernel: DENY_FROM-EXTERNAL-TO-DMZ_ICMP-REQ_IN=enp2s0 OUT=enp26s0 MAC=00:50:56:3a:e4:ba:00:0c:29:a3:9c:52:08:00 SRC=192.168.231.100 DST=192.168.89.2 LEN=84 TOS=0x00 PREC=0x00 TTL=63 ID=43191 DF PROTO=ICMP TYPE=8 CODE=0 ID=45472 SEQ=2
t id	1785264702.339880
t input.type	log

Le préfixe **DENY_FROM-EXTERNAL-TO-DMZ_ICMP-REQ_**, ajouté par le pare-feu lors de la journalisation, est bien présent dans le message. Cette observation confirme le bon fonctionnement de la chaîne de traitement, depuis la génération du journal par le pare-feu jusqu'à la création de l'alerte personnalisée et son affichage dans **Wazuh**.

Test 12 : Validation de la règle correspondant au préfixe DENY_FROM-DMZ-TO-ADMIN_ICMP-REQ_

Description : requête **ICMP** provenant de la zone **DMZ** à destination de la zone d'administration.

Le trafic est généré depuis le serveur Web, situé dans la zone **DMZ vmnet3** (**192.168.89.0/24**), afin de déclencher la règle de journalisation correspondante.

```
>>> send(IP(src="192.168.89.2", dst="172.16.44.2") / ICMP(type="echo-request", id=1000, seq=1)
Sent 1 packets.
>>>
```

Le journal produit par le pare-feu est ensuite vérifié afin de confirmer que la règle associée au préfixe **DENY_FROM-DMZ-TO-ADMIN_ICMP-REQ_** a bien été déclenchée.

```
radiolab@debian:~$ sudo tail -f -n 0 /var/log/kern.log
2026-07-28T20:48:28.556983+02:00 debian kernel: DENY_FROM-DMZ-TO-ADMIN_ICMP-REQ_IN=enp26s0 OUT=enp3s0 MAC=00:0c:29:32:e3:d8:00:0c:29:3e:56:fd:08:00 SRC=192.168.89.2 DST=172.16.44.2 LEN=28 TOS=0x00 PREC=0x00 TTL=63 ID=1 PROTO=ICMP TYPE=8 CODE=0 ID=1000 SEQ=1
```

L'événement est ensuite recherché dans **Wazuh**, depuis le module **Threat Hunting** de l'agent installé sur le pare-feu (machine **Debian**).

La description de l'alerte personnalisée associée à cette règle correspond bien à celle définie dans le fichier **local_rules.xml**.

Export Formatted	Reset view	664 available fields	Columns	Density	1 fields sorted	Full screen
timestamp	agent.name	rule.description	rule.level	rule.id		
Jul 28, 2026 @ 20:48:30.289	firewall-debian	ICMP echo-request attempt from DMZ network to admin host	6	100063		

En consultant les détails de l'événement, il est possible d'observer le message brut généré par le pare-feu.

Document Details

[View surrounding documents](#)
[View single document](#)

Table JSON

index	wazuh-alerts-4.x-2026.07.28
agent.id	001
agent.ip	172.16.44.1
agent.name	firewall-debian
decoder.name	kernel
full_log	Jul 28 18:48:28 debian kernel: DENY_FROM-DMZ-TO-ADMIN_ICMP-REQ_IN=enp26s0 OUT=enp3s0 MAC=00:0c:29:32:e3:d8:00:0c:29:3e:56:fd:08:00 SRC=192.168.89.2 DST=172.16.44.2 LEN=28 TOS=0x00 PREC=0x00 TTL=63 ID=1 PROTO=ICMP TYPE=8 CODE=0 ID=1000 SEQ=1
id	1785264510.337140
input.type	log
location	journalid

Le préfixe **DENY_FROM-DMZ-TO-ADMIN_ICMP-REQ_**, ajouté par le pare-feu lors de la journalisation, est bien présent dans le message. Cette observation confirme le bon fonctionnement de la chaîne de traitement, depuis la génération du journal par le pare-feu jusqu'à la création de l'alerte personnalisée et son affichage dans **Wazuh**.

Test 13 : Validation de la règle correspondant au préfixe **DENY_FROM-EXTERNAL-TO-DMZ_SSH-REQ_**

Description : requête **SSH** provenant de la zone externe à destination de la zone **DMZ**.

Le trafic est généré depuis la machine située dans la zone externe afin de déclencher la règle de journalisation correspondante.

```

Scapy 2.7.01
Session Actions Éditer Vue Aide
L-$ sudo scapy
INFO: Can't import PyX. Won't be able to use psdump() or pdfdump().

aSPY//YASa
  apyyyyCY////////YCa
    sY////////YSpCs  scpCY//Pp
  ayp ayyyyyySCP//Pp      syY//C
  AYAsAYYYYYYYY//Ps      cY//S
    pCCCCY//p          cSSps y//Y
    SPPPP//a          pP///AC//Y
      A//A          cyP///C
      p///Ac          sC///a
      P///YCpc          A//A
    sccccp///pSP///p      p//Y
    sY////////y caa      S//P
    cayCyayP//Ya          pY/Ya
    sY/PsY////////YCc      aC//Yp
    sc  sccaCY//PCypaapyCP//YSs
      spCPY////////YPSps
      ccaacs

Welcome to Scapy
Version 2.7.01
https://github.com/secdev/scapy
Have fun!
I'll be back.
-- Python 2

using IPython 9.11.0
Tip: You can use Ctrl-O to force a new line in terminal IPython
>>> send(
... :   IP(src="192.168.231.100", dst="192.168.89.2") / TCP(dport=22,sport=5
: 0000,flags="SA"))
.
Sent 1 packets.
>>>

```

Le journal produit par le pare-feu est ensuite vérifié afin de confirmer que la règle associée au préfixe **DENY_FROM-EXTERNAL-TO-DMZ_SSH-REQ_** a bien été déclenchée.

```

radiolab@debian:~$ sudo tail -f -n 0 /var/log/kern.log
2026-07-29T15:41:15.646038+02:00 debian kernel: DENY_FROM-EXTERNAL-TO-DMZ_SSH-REQ_IN=enp2s0 OUT=enp26s0 MAC=00:50:56:9a:e4:ba:00:0c:29:a3:9c:52:00:00 SRC=192.16
8.231.100 DST=192.168.89.2 LEN=40 TOS=0x00 PREC=0x00 TTL=63 ID=1 PROTO=TCP SPT=50000 DPT=22 WINDOW=8192 RES=0x00 ACK SYN URGP=0

```

L'événement est ensuite recherché dans **Wazuh**, depuis le module **Threat Hunting** de l'agent installé sur le pare-feu (machine **Debian**).

La description de l'alerte personnalisée associée à cette règle correspond bien à celle définie dans le fichier **local_rules.xml**.

Jul 29, 2026 @ 15:30:09.203 - Jul 29, 2026 @ 15:45:09.204				
Export Formatted Reset view 691 available fields Columns Density 1 fields sorted Full screen				
timestamp	agent.name	rule.description	rule.level	rule.id
Jul 29, 2026 @ 15:41:17.417	firewall-debian	SSH connection attempt from external to DMZ network host	8	100059

En consultant les détails de l'événement, il est possible d'observer le message brut généré par le pare-feu.

Document Details

[View surrounding documents](#)
[View single document](#)

Table JSON

f _index	wazuh-alerts-4.x-2026.07.29
f agent.id	001
f agent.ip	172.16.44.1
f agent.name	firewall-debian
f decoder.name	kernel
f full_log	Jul 29 13:41:15 debian kernel: DENY_FROM-EXTERNAL-TO-DMZ_SSH-REQ IN=enp2s0 OUT=enp26s0 MAC=00:50:56:3a:e4:ba:00:0c:29:a3:9c:52:08:00 SRC=192.168.231.100 DST=192.168.89.2 LEN=40 TOS=0x00 PREC=0x00 TTL=63 ID=1 PROT=0=TCP SPT=50000 DPT=22 WINDOW=8192 RES=0x00 ACK SYN U RGP=0
f id	1785332477.67509
f input.type	log
f location	journald
f manager.name	radlab-VMware20-1
f predecoder.hostname	debian
f predecoder.program_name	kernel
f predecoder.timestamp	Jul 29 13:41:15
f rule.description	SSH connection attempt from external to DMZ network host
# rule.firedtimes	2

Le préfixe **DENY_FROM-EXTERNAL-TO-DMZ_SSH-REQ**, ajouté par le pare-feu lors de la journalisation, est bien présent dans le message. Cette observation confirme le bon fonctionnement de la chaîne de traitement, depuis la génération du journal par le pare-feu jusqu'à la création de l'alerte personnalisée et son affichage dans **Wazuh**.

Validation de la détection d'intrusion par Suricata et des alertes Wazuh associées

Cette section a pour objectif de valider le bon fonctionnement de la chaîne de détection d'intrusion basée sur **Suricata**, ainsi que la remontée des alertes associées dans **Wazuh**.

Pour chaque scénario de test :

- Un trafic réseau correspondant à une signature de détection **Suricata** est généré depuis une machine émettrice.
- Les événements sont ensuite vérifiés dans un premier temps dans les journaux de **Suricata** (**eve.json**), puis dans l'interface **Wazuh** afin de confirmer la bonne collecte et la génération de l'alerte associée.

Test 14 : Validation de la détection Suricata d'un scan réseau via une anomalie ICMP

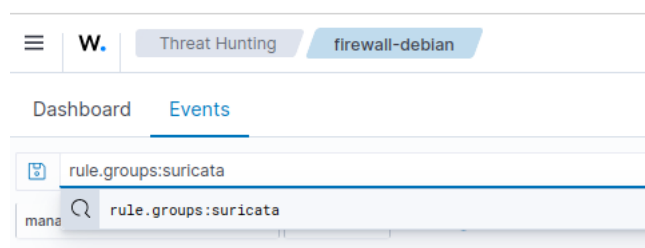
Un scan réseau est réalisé depuis la machine Kali, située dans la zone externe, à l'aide de la commande **Nmap** avec des options permettant de générer un trafic détectable par les règles de détection **Suricata**.

```
(kali@kali)~[~]
$ nmap -sV -O -A 192.168.231.1
Starting Nmap 7.99 ( https://nmap.org ) at 2026-07-28 22:05 +0200
Nmap scan report for 192.168.231.1
Host is up (0.00088s latency).
Not shown: 999 filtered tcp ports (no-response)
PORT      STATE SERVICE VERSION
80/tcp    open  http      Apache/2.4.58 ((Ubuntu))
```

Depuis le pare-feu, les journaux **Suricata** sont analysés à l'aide de la commande **jq** afin de vérifier la présence de l'événement généré dans le fichier **eve.json**.

```
radlab@debian:~$ sudo cat /var/log/suricata/eve.json | jq 'select(.event_type=="alert")'
{
  "timestamp": "2026-07-28T22:05:26.122304+0200",
  "flow_id": 1932666738605563,
  "in_iface": "enp2s0",
  "event_type": "alert",
  "src_ip": "192.168.231.100",
  "dest_ip": "192.168.231.1",
  "proto": "ICMP",
  "rule_id": "100000000",
  "rule_msg": "ICMP Echo (ping) to 192.168.231.1 from 192.168.231.100"
}
```

L'événement est ensuite recherché dans **Wazuh** en filtrant les alertes provenant de **Suricata**.



Dans la liste des événements détectés, l'alerte générée par **Suricata** apparaît correctement.

	Jul 28, 2026 @ 22:05:30.767	firewall-debian	Suricata: Alert - SURICATA ICMPv4 unknown code	  3	86601
---	-----------------------------	-----------------	--	---	-------

L'identifiant de signature (**Signature ID**) associé à cette alerte est ensuite relevé afin d'identifier la règle **Suricata** correspondante.

Document Details		View surrounding documents	View single document
agent.ip	172.16.44.1		
agent.name	firewall-debian		
data.alert.action	allowed		
data.alert.category	Generic Protocol Command Decode		
data.alert.gid	1		
data.alert.rev	2		
data.alert.severity	3		
data.alert.signature	SURICATA ICMPv4 unknown code		
data.alert.signature_id	2200025		
data.dest_ip	192.168.231.1		
data.direction	to_server		
data.event_type	alert		
data.flow.bytes_toclient	0		
data.flow.bytes_toserver	2640		
data.flow.dest_ip	192.168.231.1		
data.flow.pkts_toclient	0		
data.flow.pkts_toserver	15		
data.flow.src_ip	192.168.231.100		

Dans les règles **Suricata**, cet identifiant correspond à la règle suivante :

```
radlab@debian:~$ sudo grep 2200025 /var/lib/suricata/rules/suricata.rules
alert pkthdr any any -> any any (msg:"SURICATA ICMPv4 unknown code"; decode-event:icmpv4.unknown_code; classtype:protocol-command-decode; sid:2200025; rev:2;)
```

Cette règle détecte l'utilisation d'un code **ICMP** invalide. Plus précisément, l'alerte est déclenchée lorsqu'une combinaison **type/code ICMP** non conforme est observée.

rule.description	Suricata: Alert - SURICATA ICMPv4 unknown code
------------------	--

Ce comportement correspond généralement à une tentative de reconnaissance réseau (**scan**), bien qu'il puisse également être provoqué par un paquet **ICMP** malformé.

La combinaison observée correspond à un message **ICMP** de **type 8** avec un **code 9**.

data.icmp_code	9
data.icmp_type	8

Or, selon la spécification **ICMP** définie dans la **RFC** correspondante :

- **Type 8** : Echo Request ;
- ce type de message ne peut être associé qu'au **code 0**.

La combinaison **Type 8 / Code 9** étant inexistante dans le protocole **ICMP**, **Suricata** considère ce trafic comme anormal et génère une alerte.

September 1981

[RFC 792](#)

Echo or Echo Reply Message

0										1										2										3									
0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9	0	1								
Type										Code										Checksum																			
Identifier																				Sequence Number																			
Data ...																																							

IP Fields:

Addresses

The address of the source in an echo message will be the destination of the echo reply message. To form an echo reply message, the source and destination addresses are simply reversed, the type code changed to 0, and the checksum recomputed.

IP Fields:

Type

8 for echo message;

0 for echo reply message.

Code

0

Checksum

La signature **Suricata** identifiée dans l'événement est bien présente dans l'alerte remontée par **Wazuh**. Cette observation confirme le bon fonctionnement de la chaîne de détection, depuis l'analyse du trafic réseau par **Suricata** jusqu'à la création de l'alerte associée et son affichage dans **Wazuh**.

Test 15 : Validation de la détection Suricata avec la règle ET OPEN 210048

Afin de valider le bon fonctionnement de la détection **Suricata**, une règle de test issue du jeu de règles **Emerging Threats Open** est utilisée. Le guide de démarrage rapide de

Suricata recommande cette règle, spécialement conçue pour effectuer des tests de détection.

La règle **ET OPEN 210048** est déclenchée lors de l'accès à une **URL** de test dédiée. Pour cela, une requête **HTTP** est effectuée à l'aide de la commande **curl** :

curl http://testmynids.org/uid/index.html

Pour réaliser ce test, le pare-feu est temporairement connecté à une interface disposant d'un accès Internet afin de permettre l'accès à l'**URL** de test.

On met **Suricata** en écoute sur cette interface.

```
# Linux high speed capture support
af-packet:
- interface: enp11s0
  # Number of receive threads. "auto" uses the number of cores
  #threads: auto
  # Default clusterid. AF_PACKET will load balance packets based on flow.
  cluster-id: 99
```

Puis on génère, depuis le pare-feu, la requête avec, la commande **curl**.

```
radlab@debian:~$ sudo curl http://testmynids.org/uid/index.html
uid=0(root) gid=0(root) groups=0(root)
```

L'événement généré par Suricata est ensuite vérifié dans le fichier **eve.json** afin de confirmer le déclenchement de la règle associée.

```
radlab@debian:~$ sudo cat /var/log/suricata/eve.json | jq 'select(.alert .signature_id==2100498)'
{
  "timestamp": "2026-07-29T00:14:08.884891+0200",
  "flow_id": 19635186530265,
  "in_iface": "enp11s0",
  "event_type": "alert",
  "src_ip": "143.204.194.92",
  "src_port": 80,
  "dest_ip": "192.168.70.140",
  "dest_port": 55810,
  "proto": "TCP",
  "pkt_src": "wire/pcap",
  "tx_id": 0,
  "tx_guessed": true,
  "alert": {
    "action": "allowed",
    "gid": 1,
    "signature_id": 2100498,
    "rev": 7,
    "signature": "GPL ATTACK_RESPONSE id check returned root",
    "category": "Potentially Bad Traffic",
    "severity": 2,
    "metadata": {
      "confidence": [
        "Medium"
      ],
      "created_at": [

```

L'alerte est ensuite recherchée dans **Wazuh** afin de vérifier sa bonne collecte et son affichage.

	Jul 29, 2026 @ 00:14:10.831	firewall-debian	Suricata: Alert - GPL ATTACK_RESPONSE id check returned root	3	86601
---	-----------------------------	-----------------	--	---	-------

Document Details

[View surrounding documents](#)
[View single document](#)
[Table](#) [JSON](#)

_index	wazuh-alerts-4.x-2026.07.28
agent.id	001
agent.ip	172.16.44.1
agent.name	firewall-debian
data.alert.action	allowed
data.alert.category	Potentially Bad Traffic
data.alert.gid	1
data.alert.metadata.confidence	Medium
data.alert.metadata.created_at	2010_09_23
data.alert.metadata.signature_severity	Informational
data.alert.metadata.updated_at	2019_07_26
data.alert.rev	7
data.alert.severity	2
data.alert.signature	CDI ATTACK DESIGNED id check returned root

La signature **ET OPEN 210048** détectée par **Suricata** est bien présente dans l'événement remonté par **Wazuh**. Cette observation confirme le bon fonctionnement de la chaîne de détection, depuis le déclenchement de la règle **Suricata** lors de l'analyse du trafic réseau jusqu'à la création de l'alerte associée et son affichage dans **Wazuh**.

Validation des fonctionnalités réseau du pare-feu

Test 16 : Validation du DNAT – Accès HTTP depuis Kali vers le serveur Web

Depuis la machine **Kali**, une requête **HTTP** est envoyée vers le serveur Web en utilisant l'adresse **IP** de l'interface externe du pare-feu. La commande **wget** est utilisée pour vérifier le bon fonctionnement de la redirection d'adresse (**DNAT**).

```
(kali@kali)-[~]
$ wget 192.168.231.1
Prepended http:// to '192.168.231.1'
--2026-07-28 18:23:42-- http://192.168.231.1/
Connexion à 192.168.231.1:80... connecté.
requête HTTP transmise, en attente de la réponse... 200 OK
Taille : 10671 (10K) [text/html]
Sauvegarde en : « index.html »

index.html      100%[====>] 10,42K --.-KB/s  ds 0s
2026-07-28 18:23:42 (517 MB/s) - « index.html » sauvegardé [10671/10671]
```

Le trafic est ensuite observé sur le pare-feu. Sur l'interface externe **vmnet2 (enp2s0)**, les paquets ont bien pour destination l'adresse IP de l'interface externe du pare-feu (**192.168.231.1**).

```
radlab@debian:~$ sudo tcpdump -i enp2s0
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on enp2s0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
18:25:50.448145 IP 192.168.231.100.44438 > 192.168.231.1:http: Flags [S], seq 46317480, win 64240, options [mss 1460,sackOK,TS val 2793978031 ecr 0,nop,wscale 9], length 0
18:25:50.448732 IP 192.168.231.1:http > 192.168.231.100.44438: Flags [S.], seq 2159015150, ack 46317481, win 65160, options [mss 1460,sackOK,TS val 2187264915 ecr 2793978031,nop,wscale 7], length 0
18:25:50.449032 IP 192.168.231.100.44438 > 192.168.231.1:http: Flags [.] , ack 1, win 126, options [nop,nop,TS val 2793978032 ecr 2187264915], length 0
18:25:50.449230 IP 192.168.231.100.44438 > 192.168.231.1:http: Flags [P.], seq 1:129, ack 1, win 126, options [nop,nop,TS val 2793978032 ecr 2187264915], length 128: HTTP: GET / HTTP/1.1
18:25:50.449524 IP 192.168.231.1:http > 192.168.231.100.44438: Flags [.] , ack 129, win 509, options [nop,nop,TS val 2187264916 ecr 2793978032], length 0
```

En revanche, sur l'interface **DMZ vmnet3 (enp26s0)**, les paquets apparaissent avec pour destination l'adresse IP du serveur Web (**192.168.89.2**). Cette observation confirme que la règle de **DNAT** est correctement appliquée par le pare-feu.

```
radlab@debian:~$ sudo tcpdump -i enp26s0
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on enp26s0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
18:31:21.385905 IP 192.168.231.100.45048 > 192.168.89.2:http: Flags [S], seq 3737585436, win 64240, options [mss 1460,sackOK,TS val 2771857282 ecr 0,nop,wscale 9], length 0
18:31:21.386715 IP 192.168.89.2:http > 192.168.231.100.45048: Flags [S.], seq 4059856586, ack 3737585437, win 65160, options [mss 1460,sackOK,TS val 2187595850 ecr 2771857282,nop,wscale 7], length 0
18:31:21.387310 IP 192.168.231.100.45048 > 192.168.89.2:http: Flags [.] , ack 1, win 126, options [nop,nop,TS val 2771857284 ecr 2187595850], length 0
18:31:21.387361 IP 192.168.231.100.45048 > 192.168.89.2:http: Flags [P.], seq 1:129, ack 1, win 126, options [nop,nop,TS val 2771857284 ecr 2187595850], length 128: HTTP: GET / HTTP/1.1
18:31:21.387818 IP 192.168.89.2:http > 192.168.231.100.45048: Flags [.] , ack 129, win 509, options [nop,nop,TS val 2187595851 ecr 2771857284], length 0
```

Test 17 : Validation du SNAT – Accès HTTP depuis le serveur Web vers un service externe

Afin de valider le fonctionnement du **SNAT**, une requête **HTTP** est initiée depuis le serveur Web à destination d'un service hébergé sur la machine **Kali**.

Sur le pare-feu, une capture est réalisée sur l'interface **DMZ vmnet3 (enp26s0)**. Les paquets observés présentent comme adresse **IP** source celle du serveur Web (**192.168.89.2**), avant toute traduction d'adresse.

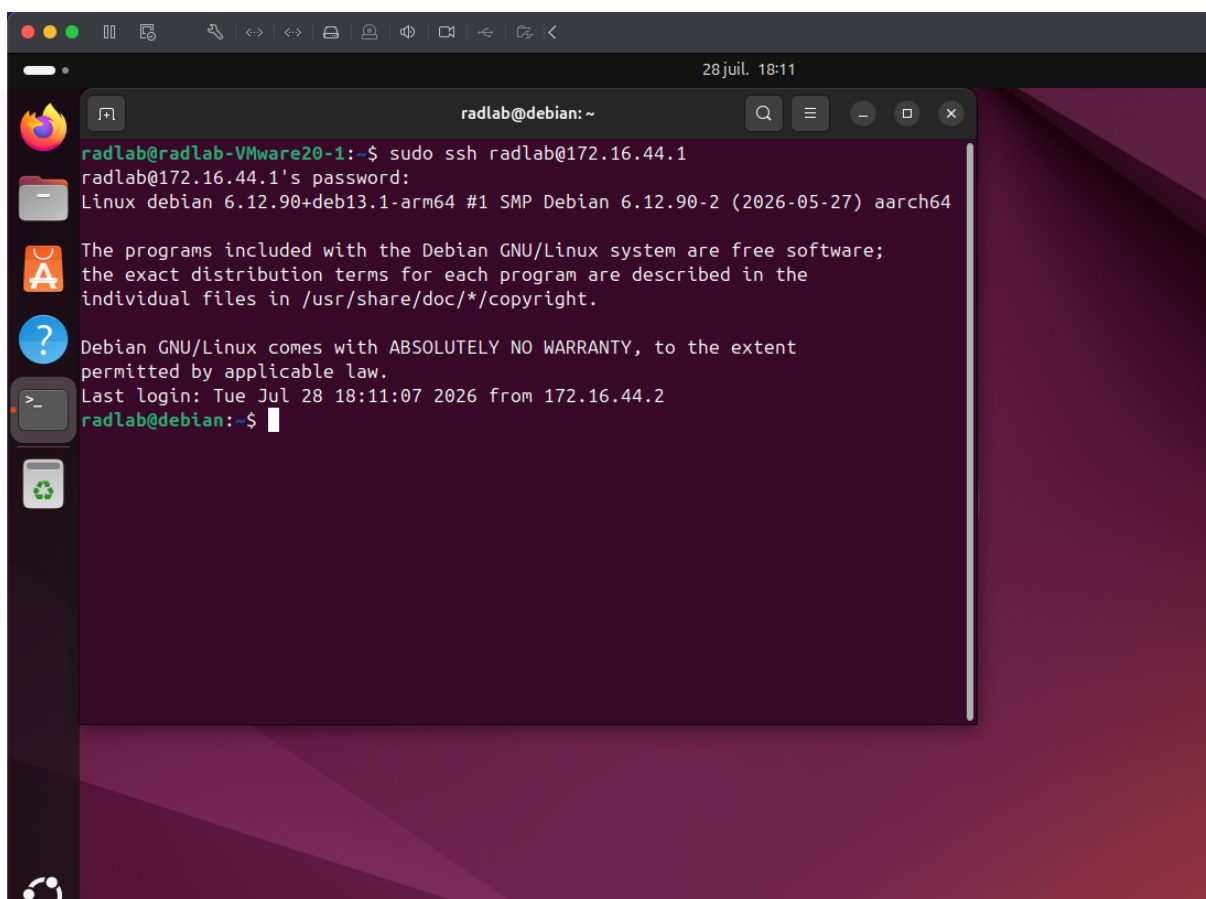
```
radlab@debian:~$ sudo tcpdump -i enp26s0
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on enp26s0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
18:49:35.200436 IP 192.168.89.2.54578 > 192.168.231.100:http: Flags [S], seq 1990107430, win 64240, options [mss 1460,sackOK,TS val 2188686694 ecr 0,nop,wscale 7], length 0
18:49:35.201325 IP 192.168.231.100.http > 192.168.89.2.54578: Flags [S.], seq 964660317, ack 1990107431, win 65160, options [mss 1460,sackOK,TS val 1238121082 ecr 2188686694,nop,wscale 9], length 0
18:49:35.201622 IP 192.168.89.2.54578 > 192.168.231.100:http: Flags [.] , ack 1, win 502, options [nop,nop,TS val 2188686696 ecr 1238121082], length 0
18:49:35.202407 IP 192.168.89.2.54578 > 192.168.231.100:http: Flags [P.], seq 1:131, ack 1, win 502, options [nop,nop,TS val 2188686696 ecr 1238121082], length 130: HTTP: GET / HTTP/1.1
18:49:35.202822 IP 192.168.231.100.http > 192.168.89.2.54578: Flags [.] , ack 131, win 128, options [nop,nop,TS val 1238121083 ecr 2188686696], length 0
18:49:35.206407 IP 192.168.231.100.http > 192.168.89.2.54578: Flags [.] , seq 1:1449, ack 131, win 128, options [nop,nop,TS val 1238121087 ecr 2188686696], length 0
```

Une seconde capture est effectuée sur l'interface externe **vmnet2 (enp2s0)**. Cette fois, les paquets ont pour adresse **IP source** celle de l'interface externe du pare-feu (**192.168.231.1**), ce qui confirme l'application de la règle de **SNAT**.

```
radlab@debian:~$ sudo tcpdump -i enp2s0
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on enp2s0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
18:51:53.306642 IP 192.168.231.1.46794 > 192.168.231.100.http: Flags [S], seq 4021076768, win 64240, options [mss 1460,sackOK,TS val 2188824800 ecr 0,nop,wscale 7], length 0
18:51:53.307102 IP 192.168.231.100.http > 192.168.231.1.46794: Flags [S.], seq 3941335581, ack 4021076769, win 65160, options [mss 1460,sackOK,TS val 1582573620 ecr 2188824800,nop,wscale 9], length 0
18:51:53.307499 IP 192.168.231.1.46794 > 192.168.231.100.http: Flags [J], ack 1, win 502, options [nop,nop,TS val 2188824801 ecr 1582573620], length 0
18:51:53.307666 IP 192.168.231.1.46794 > 192.168.231.100.http: Flags [P.], seq 1:131, ack 1, win 502, options [nop,nop,TS val 2188824801 ecr 1582573620], length 130: HTTP: GET / HTTP/1.1
```

Test 18 : Connexion SSH depuis la machine d'administration vers le pare-feu

Une connexion **SSH** est initiée depuis la machine d'administration vers le pare-feu.

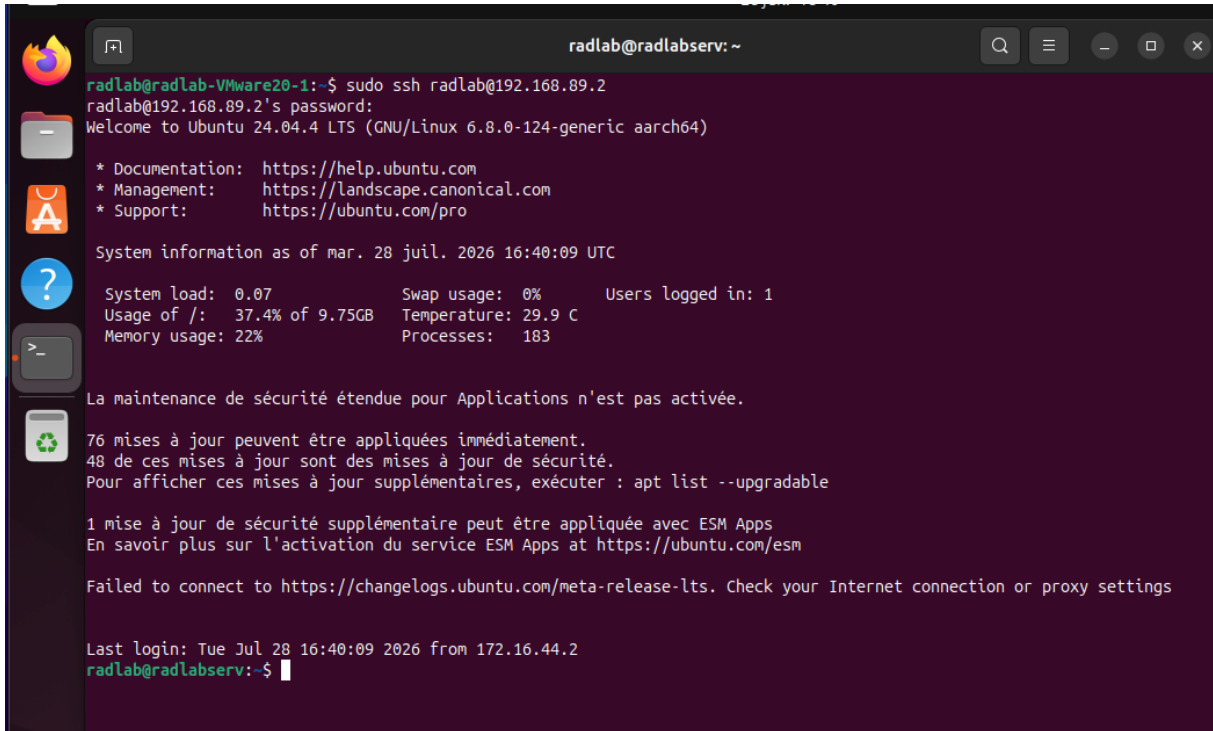


Une fois la connexion établie, celle-ci est vérifiée directement sur le pare-feu à l'aide de la commande **ss**, qui permet d'afficher les connexions réseau actives.

```
radlab@debian:~$ sudo ss | grep ssh
tcp ESTAB 0 0 172.16.44.1:ssh 172.16.44.2:54920
radlab@debian:~$
```

Test 18 : Connexion SSH depuis la machine d'administration vers le serveur Web

Une connexion **SSH** est initiée depuis la machine d'administration vers le serveur Web.



```
radlab@radlab-VMware20-1:~$ sudo ssh radlab@192.168.89.2
radlab@192.168.89.2's password:
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.8.0-124-generic aarch64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of mar. 28 juil. 2026 16:40:09 UTC

System load:  0.07               Swap usage:  0%          Users logged in: 1
Usage of /:   37.4% of 9.75GB    Temperature: 29.9 C
Memory usage: 22%               Processes:   183

La maintenance de sécurité étendue pour Applications n'est pas activée.

76 mises à jour peuvent être appliquées immédiatement.
48 de ces mises à jour sont des mises à jour de sécurité.
Pour afficher ces mises à jour supplémentaires, exécuter : apt list --upgradable

1 mise à jour de sécurité supplémentaire peut être appliquée avec ESM Apps
En savoir plus sur l'activation du service ESM Apps at https://ubuntu.com/esm

Failed to connect to https://changelogs.ubuntu.com/meta-release-lts. Check your Internet connection or proxy settings

Last login: Tue Jul 28 16:40:09 2026 from 172.16.44.2
radlab@radlabserv:~$
```

Une fois la connexion établie, celle-ci est vérifiée directement sur le **serveur Web** à l'aide de la commande **ss**.

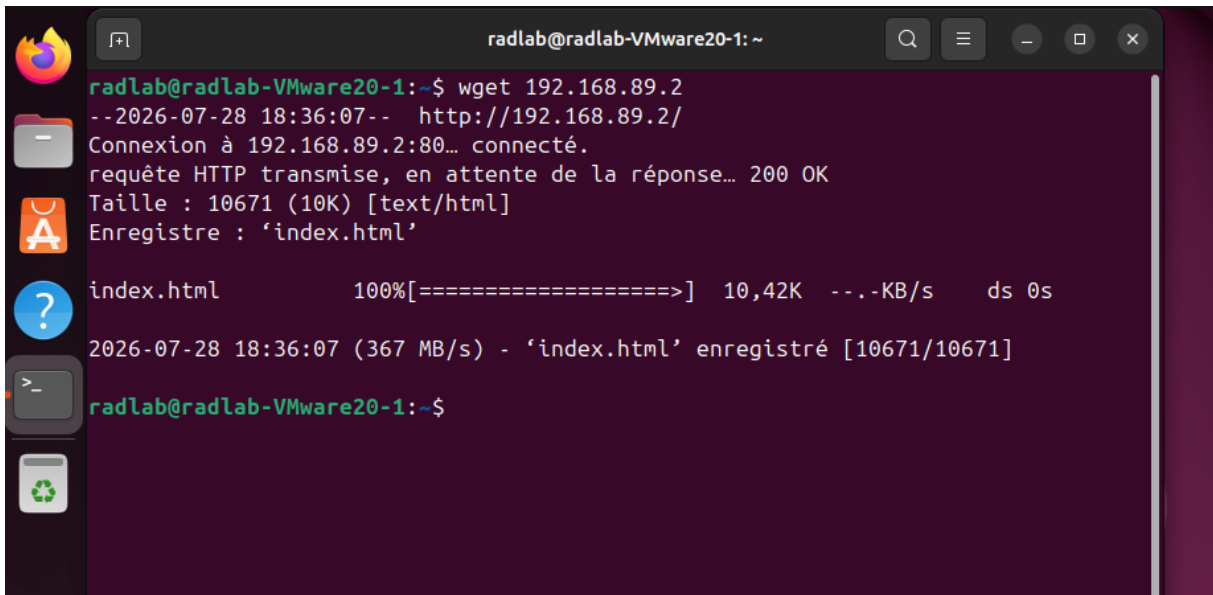


```
radlab@radlabserv:~$ sudo ss | grep ssh
tcp ESTAB 0 0 192.168.89.2:ssh 172.16.44.2:39934
radlab@radlabserv:~$
```

Test 20 : Accès HTTP depuis la machine d'administration vers le serveur Web

Depuis la machine d'administration, le **serveur Web** est directement accessible via son adresse **IP interne**. Aucun mécanisme de **DNAT** n'est appliqué aux communications provenant de la zone d'administration.

L'accès **HTTP** est testé à l'aide de la commande **wget** en ciblant l'adresse IP du serveur Web.



```

radlab@radlab-VMware20-1: ~$ wget 192.168.89.2
--2026-07-28 18:36:07-- http://192.168.89.2/
Connexion à 192.168.89.2:80... connecté.
requête HTTP transmise, en attente de la réponse... 200 OK
Taille : 10671 (10K) [text/html]
Enregistre : 'index.html'

index.html          100%[=====] 10,42K  --.-KB/s   ds 0s
2026-07-28 18:36:07 (367 MB/s) - 'index.html' enregistré [10671/10671]

radlab@radlab-VMware20-1:~$

```

Le trafic est ensuite observé sur le pare-feu à l'aide de **tcpdump**, afin de vérifier le passage de la connexion entre les deux machines.

```

n 846: HTTP
:36:07.148265 IP 172.16.44.2.51648 > 192.168.89.2.http: Flags [.], ack 7241, win 77, options [nop,r
:36:07.148265 IP 172.16.44.2.51648 > 192.168.89.2.http: Flags [.], ack 10983, win 74, options [nop,
:36:07.149128 IP 172.16.44.2.51648 > 192.168.89.2.http: Flags [F.], seq 128, ack 10983, win 77, opt
:36:07.149515 IP 192.168.89.2.http > 172.16.44.2.51648: Flags [F.], seq 10983, ack 129, win 509, op
:36:07.149700 IP 172.16.44.2.51648 > 192.168.89.2.http: Flags [F.], ack 10984, win 77, options [nop,

```